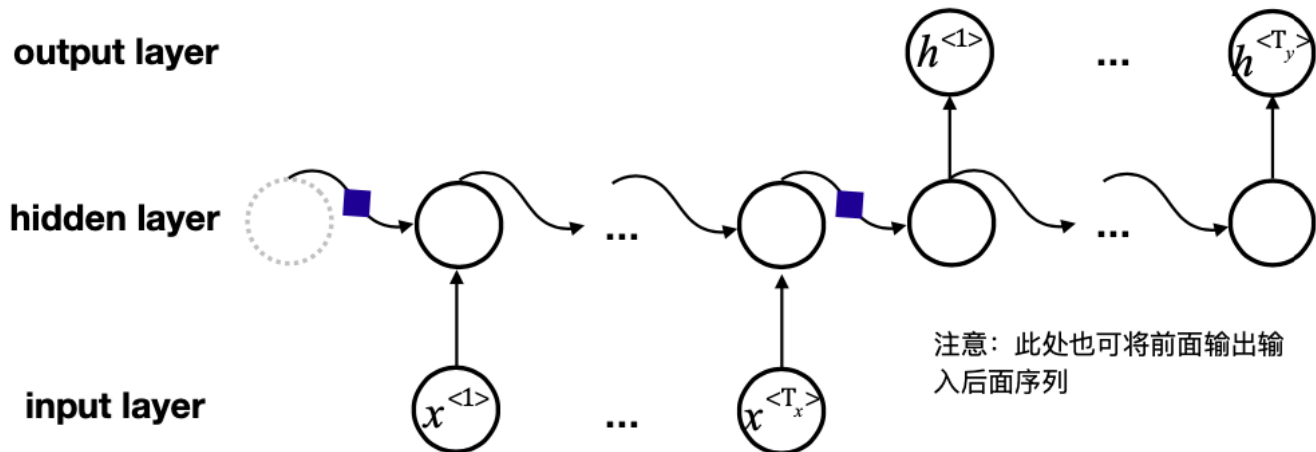


教AI的陶老师作业训练营第四期（6）

这是作业训练营的最后一次作业了，这次内容，让我们好好说一说呼声最高的"Transformer"。在具体看它的结构之前，我们需要先补充一些必备知识。

- 补充1：seq2seq

上周我们开始学习处理时序信息的模型，RNN和LSTM。我们提到了时序模型常用来解决的几种问题类型，其中一种叫做"N-M"，这次内容，我们从这种结构开始说起。如下图所示：



N-M即输入序列长度与输出序列长度不相等，例如机器翻译中将英文翻译为中文。此时需要用到多对多的RNN序列。RNN序列的终止可以根据输出特殊符号来确定，比如 $\langle \text{eos} \rangle$ 。这种模型还可以完成语音识别、文本摘要等很多任务。

在上图结构中，可以很清楚的将模型结构划分为左右两部分，左边部分用来对输入信息进行编码，右边部分对编码后的信息进行解码，这就是 `encoder` 和 `decoder`。①请将上图中 `encoder` 和 `decoder` 部分标注出来，并对其计算过程进行解释。

这种由编码器（`encoder`）和解码器（`decoder`）两部分组成，编码器接收输入序列并处理成一个向量表示，然后解码器使用这个向量生成输出序列的模型就是Seq2Seq模型(陶老师版解释，非官方)。

Seq2Seq模型的优点是可以处理长度不同的序列，成为近年来许多自然语言处理任务的标准模型。

但是，基于RNN的Seq2Seq也有一个非常致命的缺点，就是②输入过长时产生的遗忘问题，为什么？虽然在上一讲中已经了解了用来解决遗忘问题的LSTM，做了大量复杂工作是带来了效果的，但是效果有限，远不能满足需求。

当加入Attention后，显著有效的解决了遗忘问题。

- 补充2：Attention <https://arxiv.org/pdf/1409.0473v7.pdf> (<https://arxiv.org/pdf/1409.0473v7.pdf>)

就像人的注意力一样，这是一个有限的资源。

今天女朋友生气了，你把注意力全都放在了如何哄女朋友开心这件事上，所以你今天没有学习AI知识。或者你也可以分配一部分注意力在哄女朋友上面，另一部分在你的学习上。总之，你的注意力是可以分配的，同时也是有限的。模型处理信息时大概也是这样。

现在要处理的句子中t位置的信息，而前面很远地方的一个词语决定了当前t位置的词语性质，所以虽然距离远，但是应该给这个词多分配一些注意力才合理。

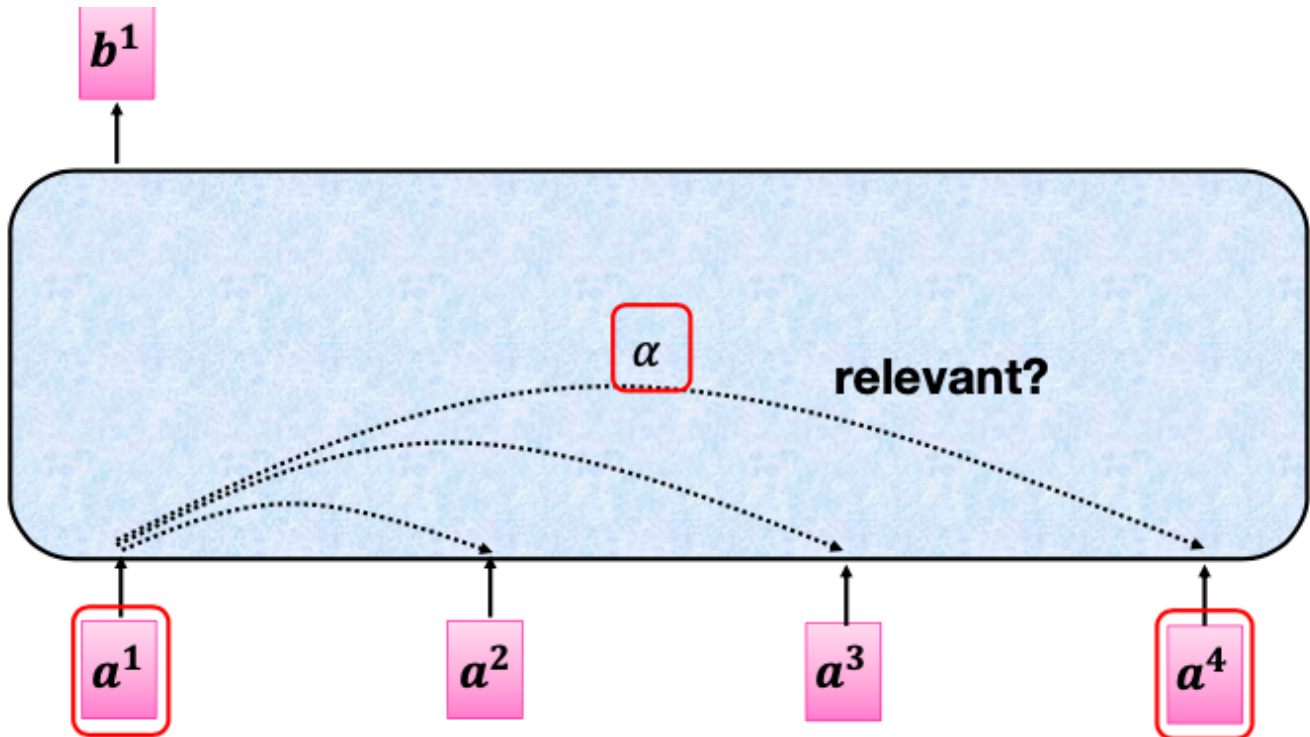
比如这句话："鲁迅先生是一位伟大的文学家和思想家，在中国文学史上占有举足轻重的地位，对中国现代文学和思想的发展影响深远，（他/她）的作品深刻地反映了社会现实和人性之间的矛盾。"

括号里当然应该选择"他"，因为你记得前面说的是"鲁迅先生"，但是由于RNN的遗忘性，这么远的距离就可能会忘掉，而Attention会分配给"鲁迅先生"一个较高的权重，这样就避免了因为距离而产生的遗忘问题。

③参照论文，以上图"N-M"示意图为例，画图解释加上Attention后是如何计算的。

④这里面是如何体现注意力的分配的？

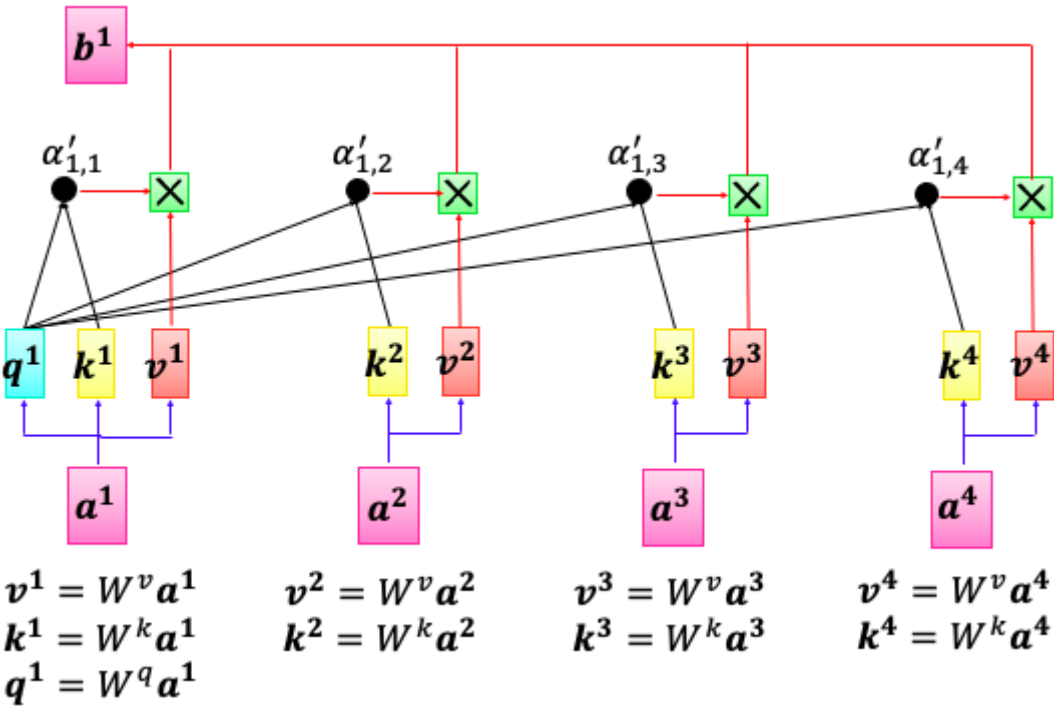
- 补充3：Self-Attention <https://arxiv.org/pdf/1601.06733v7.pdf> (<https://arxiv.org/pdf/1601.06733v7.pdf>)



现在这个模式是"N-N",有多少个输入就有多少个输出。 b_1 是 a_1 对应的输出(b_2 、 b_3 、 b_4 都有只是这里没有画出来)，它的获得不是只看 a_1 就能得到的，而是将所有输入都看完再进行输出，但是会分配给不同位置的输入信息不同的注意力分数 α 。(注意1：由于这个部分可以叠加，现在的输入有可能是前一层的输出，所以这里用 a 表示而没有用 x 。注意2：这里用英文字母 a 表示输入用希腊字母 α 「读作alpha」表示注意力分数。)

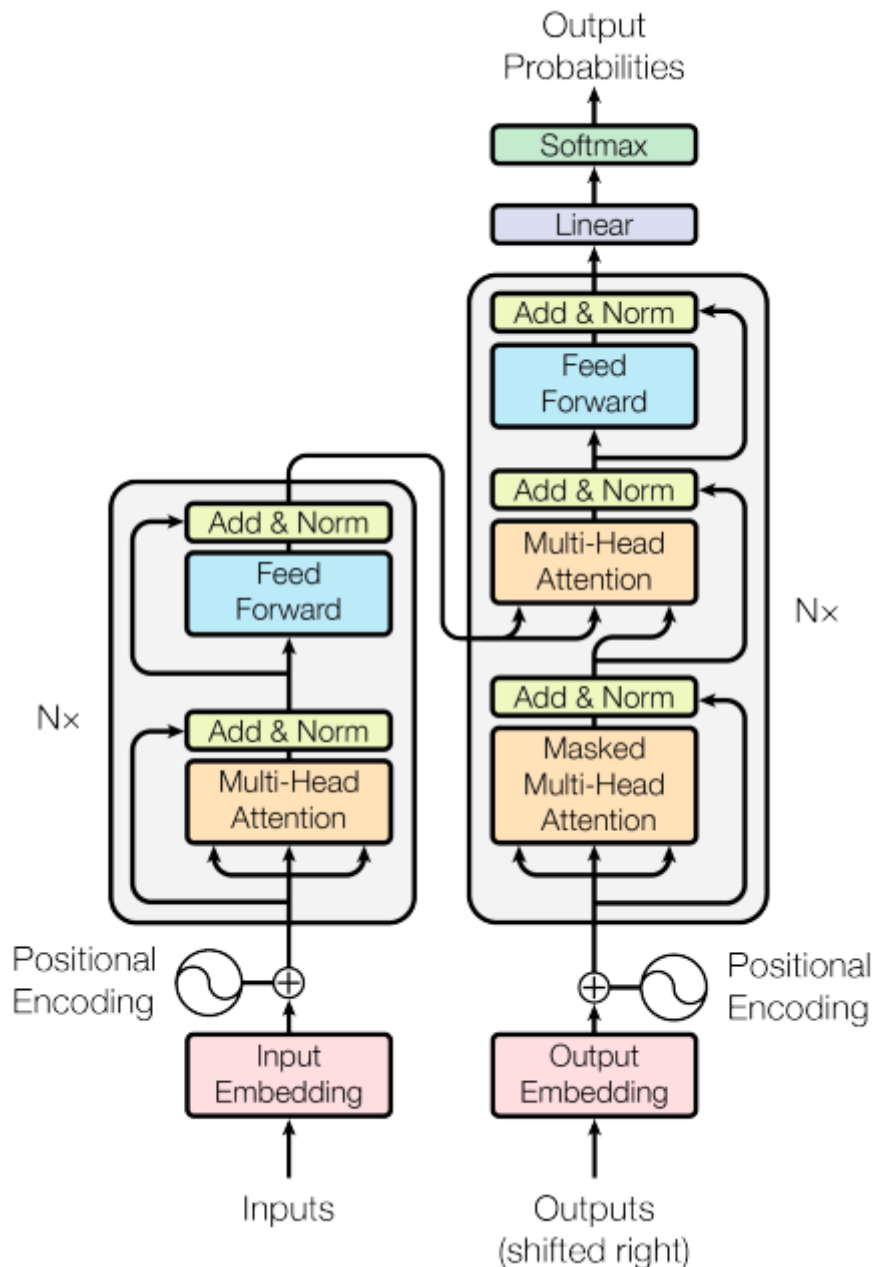
由获得的 α 还要再经过softmax得到 α' 。

得到注意力分数后，便可以根据注意力分数提取信息：



- ⑤ b_1 是如何计算求得的？
- ⑥ 类比来看， b_2 应当如何计算求得？
- ⑦ 如何理解计算过程中出现的 q 、 k 、 v ？
- ⑧ 上图中，参数有哪些？有多少？
- ⑨ 如何理解 b_1 、 b_2 、 b_3 、 b_4 的并行计算过程？用公式证明。
- ⑩ 多个上述的自注意力拼在一起就是多头自注意力，多头自注意力具体是怎样计算的？

终于做完了铺垫，现在可以来看Transformer的结构了，简单来说，它就是一个基于多头自注意力的seq2seq模型。



这个结构图清晰的划分为左右两个部分，分别就是 encoder 和 decoder。不过要想真正把这个结构搞清楚，还得继续弄明白下面几个问题：

- ⑪ 位置编码是什么？这里加入位置编码有什么意义？
- ⑫ Add 在这里是怎么计算的？有什么作用？
- ⑬ Norm 在这里的作用是什么？这里的 Norm 是如何计算的？
- ⑭ 多头自注意力前加了个 Masked 是怎么回事？为什么这么设计？
- ⑮ Feed Forward 是什么设计？
- ⑯ encoder 和 decoder 之间是如何进行信息传递的？
- ⑰ 原论文中 N 是几？

这些问题都搞定了，这个结构也就没问题了，这应该也不是太简单，原论文给你参考

<https://arxiv.org/pdf/1706.03762v5.pdf> (<https://arxiv.org/pdf/1706.03762v5.pdf>)。也可以从矩阵计算的角度上来理解模型结构。

对于Transformer，PyTorch2 中已经将模型嵌入，可以通过 `torch.nn.Transformer` 直接来使用了，当然还需要设置对应的参数，使用说明文档：<https://pytorch.org/docs/stable/generated/torch.nn.Transformer.html> (<https://pytorch.org/docs/stable/generated/torch.nn.Transformer.html>)

或者，如果你有兴趣的话，也可以自己去试着搭建Transformer。

最后一次作业了，把我想说的都说了吧。其实现在最受关注的大语言模型当然还是GPT，不过你知道吗，它就是把Transformer的Decoder单独拿出来训练的。单独拿Encoder来训练模型的也有，这就是另一个大名鼎鼎的语言模型BERT。其实BERT的设计和训练更巧妙，但GPT胜在个头大，目前看效果更好些。有兴趣的同学自己再去深入研究吧。

你以为作业到这就结束了？当然不能。

最后一次作业了，把我想让你们做的也都留了吧。

1. 图像分类练习：

⑩尝试搭建CNN模型，使用给定数据集实现情绪识别。

2. 时序信息练习：

⑪选择网络实现电力负荷预测。

3. 语言模型练习：

⑫选择生成一个"李白"或"鲁迅"风格的文本生成模型。(这多少有点难度了，有兴趣的同学可以尝试一下)

最后的最后：

⑬如果可以的话，我想听到您的反馈：收获、困难、评价、建议等等，感谢。

感谢同学们一路陪伴，希望六周的学习能给你带来收获。